

LLM-Based Generative Retrieval for Snapchat Content Recommendation

Liam Collins*, Jiwen Ren*, Donald Loveland, Bhuvish Kumar, Clark Mingxuan Ju, Xuan Guo, Mo Li, Alvin Hou, Yi Cui, Peng Yang, Jian Wang, Saud Afzal Shafi, Nga Than, Ruiming Lu, Wenfeng Zhuo, Dongheng Li, Lili Zhang, Mingtao Zhang, Jinchao Ye, Vincent Xue, Chunhui Zhu, Neil Shah
Snap, Inc.
USA
{lcollins2, jren2, nshah}@snapchat.com

Abstract

Pretrained large language models (LLMs) are promising retrieval engines because they combine rich semantic priors, strong sequence modeling capabilities, and favorable scaling behavior. However, turning a pretrained LLM into a generative retriever in production deployment raises several challenges: the model must learn an internal item vocabulary that was absent from pretraining, and generate valid item identifiers under strict latency and cost constraints. We address these challenges through the design and launch of SnapLGR, an LLM-based generative retrieval system for short-video recommendation at Snapchat. The system is built around three main designs. First, we construct semantic identifiers (SIDs) from multimodal item embeddings and enhance them with Personalized PageRank (PPR)-based co-engagement contrastive learning, resulting in improved codebook utilization, reduced collisions, and infused collaborative signal. Second, we use continued pretraining (CPT) to ground the introduced SID tokens before supervised fine-tuning (SFT) on user interaction sequences. Third, we make SnapLGR serving practical through TensorRT-LLM CUDA-backed beam search and a decentralized worker-loop architecture. In a live A/B test, the launched system increased View Time by 0.37%, Time Spent by 0.09%, Deep Sessions by 0.18%, and Deep Sessions Unique User by 0.11% relative to the existing TIGER-style generative retrieval baseline. We then decompose this offline gap under a fixed tokenizer and quantify the gains due to model architecture, scaling, and pretraining. Overall, our deployment shows that successful production SnapLGR requires joint design across representation learning, vocabulary grounding, and efficient training and serving.

CCS Concepts

• **Information systems** → **Recommender systems**; *Retrieval models and ranking*; • **Computing methodologies** → *Natural language processing*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/27/08
<https://doi.org/XXXXXXX.XXXXXXX>

Keywords

generative retrieval, semantic IDs, LLM recommendation, production systems, inference optimization, continued pretraining

ACM Reference Format:

Liam Collins*, Jiwen Ren*, Donald Loveland, Bhuvish Kumar, Clark Mingxuan Ju, Xuan Guo, Mo Li, Alvin Hou, Yi Cui, Peng Yang, Jian Wang, Saud Afzal Shafi, Nga Than, Ruiming Lu, Wenfeng Zhuo, Dongheng Li, Lili Zhang, Mingtao Zhang, Jinchao Ye, Vincent Xue, Chunhui Zhu, Neil Shah. 2027. LLM-Based Generative Retrieval for Snapchat Content Recommendation. In . ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large-scale recommendation systems rely heavily on retrieval to efficiently filter massive item corpora for downstream ranking. Generative retrieval (GR) reformulates this task as sequence generation: given a user’s interaction history, a model generates semantic IDs (SIDs)—discrete token sequences capturing item semantics—for the next items the user is likely to engage with. Following TIGER [27], initial GR systems have relied on small, randomly initialized encoder-decoder transformers such as T5 [26] as the backbone SID generation model. Nevertheless, replacing these backbones with pretrained decoder-only large language models (LLMs) offers immense promise. LLMs bring rich semantic priors, advanced sequence modeling capabilities, and predictable scaling [10, 14], driving significant industrial interest in LLM-based GR [4, 5, 7, 9, 11, 31–33, 36].

However, transitioning an LLM into an active generative retriever for a large-scale production system introduces coupled modeling and infrastructure challenges:

- (1) **Tokenizer Design:** SIDs are the interface between the retriever and the item corpus, thus are critical to GR performance [13]. The challenge lies in designing a tokenizer that effectively compresses semantic features, and potentially collaborative signals as well, into SIDs, while avoiding severe token collisions.
- (2) **Vocabulary Grounding:** SIDs do not exist in the LLM’s pre-training vocabulary. The model must semantically ground these new tokens into its existing internal knowledge space before it can accurately leverage its pretrained semantic understanding for recommendation.
- (3) **High-Throughput Training and Serving:** The system must regularly train and execute wide-beam inference at large scale all within a strict GPU budget in spite of LLMs’ pronounced computational complexity.

*Indicates equal contribution.

Despite growing interest in LLM-based GR, it is still not well-understood how to jointly tackle these deployment challenges. Moreover, despite the promise of LLM-based GR, its performance gains over legacy TIGER-style GR systems at production scale remain unclear. This raises two critical questions: (1) How should we build an LLM-based generative retrieval system to overcome the aforementioned production challenges? and (2) What are the tangible online business gains and the underlying sources of offline performance improvements of this system over legacy TIGER-style baselines?

We address these questions through the design, production launch, and empirical analysis of **SnapLGR: Snapchat Language-based Generative Retrieval**, an end-to-end LLM-based GR system for short video recommendation at Snapchat, a widely-used social media platform. Our main contributions are:

- **Multimodal, Behavioral SIDs:** We build SIDs by residually quantizing Qwen3-VL [19] multimodal embeddings. We further introduce a Personalized PageRank (PPR) [8] co-engagement contrastive loss that pulls co-engaged items together in the SID space, infusing collaborative signal and reducing collisions.
- **Two-Stage Vocabulary Grounding:** We achieve semantic grounding by decoupling vocabulary learning from task tuning. A frozen-LLM Continued Pretraining (CPT) stage first anchors new SID tokens in textual semantics, followed by Supervised Fine-Tuning (SFT) on user interaction histories.
- **High-Throughput Training and Inference:** We co-optimize our infrastructure to support frequent periodic retraining and massive-scale batch inference. For training, we achieve a 3.63 $\times$ throughput increase via tuned sequence packing, FlashAttention-2 [3] variable-length kernels, and graph compilation. For inference, we eliminate Python framework bottlenecks using TensorRT-LLM's [23] CUDA-backed beam search, a decentralized worker-loop architecture, and asynchronous I/O, delivering roughly 45.7 $\times$ speedup in per-GPU throughput over our legacy baseline.
- **Production Launch:** In a 7-day live A/B test, our launched system achieved a +0.37% increase in View Time, a +0.09% increase in Time Spent, a +0.18% increase in Deep Sessions, and a +0.11% increase in Deep Sessions Unique User over the incumbent TIGER-style GR baseline.
- **Offline Analysis:** We conduct a series of offline studies to understand the reasons for SnapLGR's improvement over the legacy TIGER-style system, finding that the decoder-only backbone and model scaling contribute most significantly to the gain.

SnapLGR is currently deployed in production as a retrieval source for a short-form video retrieval platform at Snapchat. Overall, our successful launch demonstrates that industrial-scale generative retrieval is an end-to-end co-design problem, requiring close alignment across item representation, vocabulary grounding, and training and inference engineering.

2 System Design

SnapLGR comprises four interdependent components: a video tokenizer that encodes videos as SIDs (§2.1); a two-stage LLM training pipeline (§2.2); an inference pipeline including SID-to-video materialization, which connects generation to the serving stack (§2.3); and high-throughput training and inference systems (§2.4). We provide additional implementation details in Appendix B.

2.1 Multimodal, Behavior-aware SIDs

Semantic IDs (SIDs) are the canonical item representation in GR. SIDs assign multiple tokens per item, which allows for exponentially reducing embedding table dimensions, caters to the sequence modeling and generation capabilities of GR backbones, and enables encoding cross-item relationships via shared tokens. Formally, the SID for items $i \in \mathcal{I}$ has L levels. Its code at level ℓ is $z_i^\ell \in \{1, \dots, K_\ell\}$, where K_ℓ is the size of the corresponding codebook; we denote the complete SID by $\mathbf{z}_i = (z_i^1, \dots, z_i^L)$.

An effective tokenizer for production video retrieval must map items to discrete tokens in a way that captures both rich multimodal semantics and collaborative user engagement signals, allowing the generative backbone to effectively learn and leverage cross-video relationships. At the same time, the tokenizer must be heavily collision-resistant, dispersing video assignments broadly across the code space to prevent large clusters of distinct videos from collapsing into identical SIDs. This ensures the generated codes maintain fine representation granularity and simplifies downstream SID-to-video materialization over a large-scale corpus. To achieve this, our construction combines a multimodal embedding, residual quantization, and graph-derived co-engagement supervision.

Multimodal embedding. Let v_i and t_i denote the video and text description for video i , where the text description is generated by Gemini 2.5 Flash [2]. To capture a strong baseline of item semantics, we use a powerful multimodal embedding model, Qwen3-VL-Embedding-8B [19], to map these features to a joint representation $\mathbf{e}_i = \phi(v_i, t_i)$.

Residual quantization. After embedding each video, we train a residual quantizer to map these embeddings to SIDs, following the RQ-VAE design of Ju et al. [13]. An encoder E maps the normalized embedding to an initial residual of width d_q , $\mathbf{r}_i^1 = E(\text{LN}(\mathbf{e}_i))$, where we apply LayerNorm (LN) to stabilize training and prevent codebook collapse. Each level has a codebook $\mathbf{C}_\ell \in \mathbb{R}^{K_\ell \times d_q}$. At level ℓ , we compute cosine similarity scores $s_{ic}^\ell = \text{cossim}(\mathbf{r}_i^\ell, \mathbf{C}_\ell[c])$ against every code $c \in \{1, \dots, K_\ell\}$, then subtract the code with the max score:

$$\mathbf{r}_i^{\ell+1} = \mathbf{r}_i^\ell - \mathbf{C}_\ell[z_i^\ell], \quad \text{where} \quad z_i^\ell = \arg \max_{c \in \{1, \dots, K_\ell\}} s_{ic}^\ell. \quad (1)$$

Thus the first code approximates the encoder output, and later codes successively encode what earlier levels did not capture.

We use the hard-forward, full-codebook straight-through estimator from Ju et al. [13] to pass gradients through the discrete $\arg \max$ in Equation (1). Let $\mathbf{s}_i^\ell = (s_{i1}^\ell, \dots, s_{iK_\ell}^\ell)$, let $\mathbf{a}_i^\ell = \text{softmax}(\mathbf{s}_i^\ell)$, and let $\text{sg}(\cdot)$ denote the stop gradient operation. During training, the selected vector is represented as:

$$\tilde{\mathbf{q}}_i^\ell = \mathbf{C}_\ell[z_i^\ell] + (\mathbf{a}_i^\ell)^\top \mathbf{C}_\ell - \text{sg}((\mathbf{a}_i^\ell)^\top \mathbf{C}_\ell). \quad (2)$$

The last two terms cancel in the forward pass, so the SID remains a hard assignment. In the backward pass, however, they provide gradients through the similarity scores and all codebook rows, reducing the risk that unselected codes remain permanently unused. The decoder reconstructs the normalized input as $\tilde{\mathbf{x}}_i = D(\text{LN}(\sum_{\ell=1}^L \tilde{\mathbf{q}}_i^\ell))$. For a minibatch of N video embeddings, the RQ-VAE loss consists

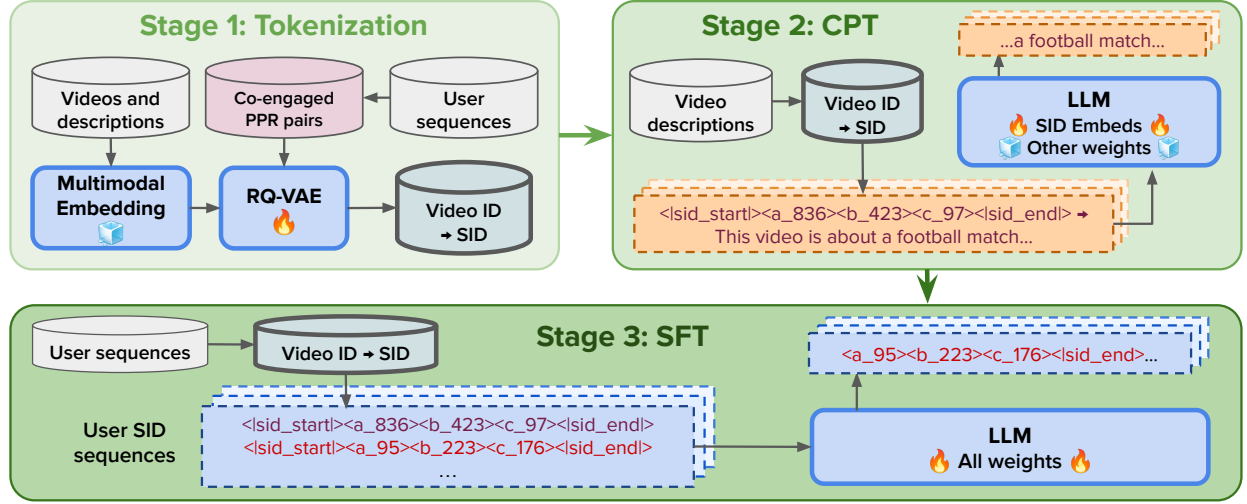


Figure 1: SnapLGR Training pipeline. *Stage 1: Tokenization* (§2.1). We train an RQ-VAE on multimodal Qwen3-VL embeddings with PPR-based co-engagement supervision to produce the video-to-SID map. *Stage 2: CPT* (§2.2.1). We ground newly added SID embeddings in the LLM’s vocabulary using video text descriptions. *Stage 3: SFT* (§2.2.2). We fine-tune the full LLM to predict future engaged SIDs from users’ SID interaction histories.

of reconstruction and commitment losses:

$$\mathcal{L}_{\text{rec}} = \frac{1}{Nd} \sum_{i=1}^N \|\text{LN}(\mathbf{e}_i) - \hat{\mathbf{x}}_i\|_2^2, \quad (3)$$

$$\mathcal{L}_{\text{com}} = \frac{1}{2NLd_q} \sum_{i=1}^N \sum_{\ell=1}^L \left(\|\mathbf{r}_i^\ell - \text{sg}(\mathbf{q}_i^\ell)\|_2^2 + \|\text{sg}(\mathbf{r}_i^\ell) - \mathbf{q}_i^\ell\|_2^2 \right), \quad (4)$$

where $\mathbf{q}_i^\ell = \mathbf{C}_\ell[z_i^\ell]$. The two commitment terms pull encoder residuals toward their assigned codes and update code vectors toward assigned residuals, respectively.

Co-engagement contrastive loss. Following PLUM [9], we additionally inject collaborative signal into the SIDs via a co-engagement-based contrastive loss to improve their behavioral alignment and recommendation utility. We implement this by first feeding the video’s sum of quantized latents through a projection head G_c :

$$\mathbf{g}_i = \text{normalize} \left(G_c \left(\sum_{\ell=1}^L \tilde{\mathbf{q}}_i^\ell \right) \right). \quad (5)$$

For a minibatch of co-engaged video pairs $\{(i_p, i_p^+)\}_{p=1}^B$, we use i_p^+ as the positive for anchor i_p and the other $B-1$ positive-side videos as in-batch negatives:

$$\mathcal{L}_{\text{co}} = -\frac{1}{B} \sum_{p=1}^B \log \frac{\exp(\mathbf{g}_{i_p}^\top \mathbf{g}_{i_p^+} / \tau)}{\sum_{p'=1}^B \exp(\mathbf{g}_{i_p}^\top \mathbf{g}_{i_{p'}^+} / \tau)}. \quad (6)$$

Here $\tau > 0$ is the contrastive temperature. The complete tokenizer objective is $\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda_{\text{com}} \mathcal{L}_{\text{com}} + \lambda_{\text{co}} \mathcal{L}_{\text{co}}$.

PPR positive-pair construction. To construct the closely related positive video pairs $\{(i_p, i_p^+)\}$ required for Equation (6), we build an undirected bipartite user–video graph $\mathcal{G} = (\mathcal{U} \cup \mathcal{V}, \mathcal{E})$ from eligible engagements. We compute Personalized PageRank (PPR) [8] scores over the two-hop neighborhood for each anchor video i (detailed in Appendix B.1.1). Because the graph is bipartite, a two-step

random walk connects the anchor to another video j through a shared user, naturally accounting for multiple overlapping interaction paths while avoiding bias toward globally popular nodes. We rank video nodes $j \neq i$ by their PPR score and retain high-scoring (i, j) pairs as contrastive positives. In production, we use the P75 score across all pairs as the minimum threshold for selection.

2.2 LLM Training Pipeline

After tokenizing the videos, we need to teach the backbone LLM to recommend over the SID vocabulary. We do this in two stages: continued pretraining (CPT) and supervised fine-tuning (SFT). We use Qwen3-0.6B [25] as the backbone LLM.

2.2.1 Continued Pretraining (CPT). The first step of CPT is to append the new SID tokens to the LLM’s vocabulary. We randomly initialize their embeddings from a Gaussian distribution with mean and covariance equal to those of the existing embeddings. Then, we align the SID embeddings by training them on a video SID-to-description task. In particular, for each video i , we get its SID $\mathbf{z}_i$ and text description t_i , prompt the LLM to generate t_i given $\mathbf{z}_i$, and compute the loss over the generated text description tokens. We train only the SID embeddings on this loss while keeping the rest of the model frozen, serving to ground the embeddings in the LLM’s existing world knowledge. We refer the reader to Appendix B.2.1 for an illustrative prompt.

2.2.2 Supervised Fine-Tuning (SFT). SFT fine-tunes the LLM checkpoint produced by CPT on users’ SID interaction sequences. We use the ChatML prompt format where the user prompt contains the chronological SID history and the assistant prompt has future events, and we compute the loss only on the assistant tokens.

2.2.3 Training Lifecycle. CPT and SFT operate on distinct schedules in our production pipeline. CPT is executed once during the

initial base model training phase over a broad corpus of video descriptions to firmly establish the new SID vocabulary. Following this, SFT transitions into hourly incremental training on fresh interaction sequences to maintain temporal relevance.

2.3 Inference Pipeline

Figure 2 illustrates our end-to-end serving pipeline, which is structurally divided into offline batch generation and online serving stages. In the offline stage, historical user SID sequences are daily by a TensorRT-LLM batch inference engine to generate predicted future SIDs. Because SIDs represent semantic clusters rather than exact items, they pass through a value-weighted materialization step using a precomputed SID-to-Video ID mapping. The resulting user-to-video candidate lists are written to a low-latency serving index. During the online stage, a user request triggers the retrieval component to fetch these pre-materialized video candidates directly from the serving index. Finally, these candidates are passed downstream to the ranking layer to assemble the final response.

2.4 Throughput Optimizations

Incremental SFT retraining and batch inference over hundreds of millions of sequences requires tailored optimization under strict GPU budgets. Standard general-purpose LLM infrastructure is poorly suited for generative retrieval: training suffers from sequence padding inefficiencies, while inference is bottlenecked by Python runtime overheads during short-sequence wide-beam decoding. We make several optimizations to our training and serving pipelines to maximize hardware utilization and throughput.

Training. We apply a cumulative series of software optimizations:

- (1) *torch.compile*: We leverage graph compilation to reduce Python framework overhead and fuse basic GPU operations.
- (2) *FlashAttention-2 Variable-Length Support*: We utilize FlashAttention-2's built-in variable-length capabilities to compute attention directly over unpadded sequences.
- (3) *Dynamic Sequence Packing*: We concatenate user interaction histories into contiguous memory buffers, eliminating wasted computation on padding tokens.

As we discuss in §4.4.2, this optimization stack delivers a 3.63× end-to-end throughput increase over the unoptimized baseline when combined with a hardware migration from NVIDIA A100 to NVIDIA H100 GPUs.

Inference. Our serving infrastructure is built around three core designs aimed at achieving high-throughput SID generation:

- (1) *CUDA-Backed Beam Search*: We execute generation using TensorRT-LLM's [23] CUDA-backed runtime. This ensures beam expansion, scoring, and KV-cache management occur entirely on-GPU, avoiding redundant device-to-host memory copies and minimizing Python scheduling overhead.
- (2) *Decentralized Worker-loop Architecture*: We employ a decentralized worker-loop processing model. Each worker independently claims, loads, and processes its assigned Parquet data shards, fully removing centralized driver-node coordination and fan-out bottlenecks from the critical path.
- (3) *Asynchronous I/O Overlapping*: To prevent data ingestion and output serialization from stalling GPU execution, we implement background row-group prefetching and isolate Parquet

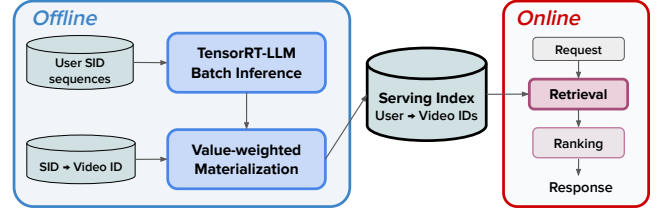


Figure 2: Serving pipeline. *Offline*: daily batch inference generates each user’s top- k SIDs. Value-weighted materialization maps the SIDs to candidate videos and writes user-video lists to an index. *Online*: given a request, the serving index serves the video ID list as retrieval candidates.

writes to a dedicated asynchronous thread pool. Additionally, we strictly gate non-essential CPU operations, such as unused metric encoding, out of the generation loop.

Ultimately, compounding these architectural changes and I/O overlapping strategies yields end-to-end per-GPU throughput gains of 45.7× over the legacy un-optimized infrastructure. We detail the quantitative impact of each specific optimization in §4.4.2.

3 SID Quality and CPT Grounding Evaluation

We begin by evaluating the pre-SFT components of our pipeline offline. In §3.1, we evaluate the tokenizer’s representation quality with a focus on *collision resistance*. In §3.2, we evaluate how effectively continued pretraining (CPT) grounds the newly introduced SID tokens into the LLM’s vocabulary.

3.1 SID Quality & Collision Resistance

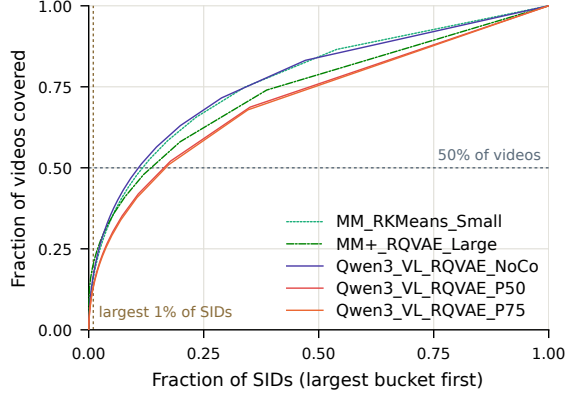
An unbalanced code mapping results in severe token collisions, where disproportionately large sets of stories map to identical SIDs. This degrades the LLM’s ability to distinguish subtle semantic differences between items and complicates downstream candidate materialization.

Metrics. We evaluate collision resistance using four metrics: *utilization* (the proportion of the theoretical code space occupied), *uniqueness* (the fraction of items assigned a unique SID), *top-1% coverage* (the fraction of stories falling into the largest 1% of SID buckets), and the *Gini coefficient* of the collision distribution (summarizing the concentration curve in Figure 3). Lower Gini and top-1% coverage values indicate that stories are more evenly dispersed across SIDs. Formal metric definitions are in Appendix C.1.

Baselines. We compare our production tokenizer that uses Qwen3-VL embeddings and PPR co-engagement supervision above P75, denoted **Qwen3_VL_RQVAE_P75**, against four baselines evaluated on the same large-scale video cohort: **MM_RKMeans_Small**, which uses internal multimodal embeddings and residual KMeans [12]; **MM+_RQVAE_Large**, a similar internal baseline that concatenates additional multimodal embeddings and uses RQ-VAE; and two Qwen3_VL_RQVAE ablations, **NoCo** without co-engagement, and **P50** with a lower engagement threshold. All versions use codebook widths (1024, 512, 256) besides **MM_RKMeans_Small** which uses (256, 256, 256).

Table 1: SID assignment statistics.

SID	Util.%	Uniq.%	Gini	Top-1%
MM_RKMeans_Small	36.3	15.8	0.548	16.0
MM+_RQVAE_Large	45.5	29.1	0.483	20.4
Qwen3_VL_RQVAE_NoCo	31.8	16.8	0.551	16.9
Qwen3_VL_RQVAE_P50	48.2	31.3	0.432	13.6
Qwen3_VL_RQVAE_P75	49.1	32.1	0.424	13.1

**Figure 3: SID collision concentration. SIDs are ordered by decreasing bucket size. Lower curves indicate a more balanced assignment; dashed guides mark the largest 1% of SIDs and 50% video coverage.**

Results. As shown in Table 1, PPR co-engagement supervision dramatically improves code utilization and collision resistance. Within the Qwen3-VL cohort, adding co-engagement at the P50 threshold increases utilization from 31.8% to 48.2% and nearly doubles uniqueness (16.8% to 31.3%), with P75 filtering providing marginal further gains. Figure 3 illustrates this improved concentration visually, showing that the Qwen3-VL co-engagement variants display curves closer to the diagonal than the baselines.

3.2 SID Grounding

When new SID tokens are appended to the LLM vocabulary, their embeddings are typically initialized as the mean of existing text tokens with minor variance, meaning they lack alignment with the LLM’s semantic knowledge space, i.e. they are ungrounded. Here we evaluate how well CPT achieves that grounding.

Metrics. We use **Text-Grounding RSA** (Representational Similarity Analysis) [15] to evaluate SID grounding. RSA quantifies the geometric correlation between the learned SID embeddings and the LLM’s text embeddings of the corresponding item descriptions.

Formally, for a set of N items, let $\mathbf{t}_i$ denote the mean-pooled base text embedding of item i ’s description, and $\mathbf{s}_{i,\ell}$ denote the learned embedding of item i ’s ℓ -th SID token. For each layer $\ell \in [L]$, we construct two $N \times N$ pairwise cosine distance matrices, $D^{\text{text},\ell}$ and $D^{\text{SID},\ell}$, where elements are defined as $D_{i,j} = 1 - \cos(\mathbf{x}_i, \mathbf{x}_j)$. The Text-Grounding RSA is the Spearman rank correlation ρ between the flattened strictly upper-triangular elements of these matrices,

Table 2: SID Grounding Evaluation. Post-CPT embeddings yield the strongest proxy for text semantic alignment, though this alignment is mostly lost after SFT.

Probe	RSA $\uparrow$
Random Initialization	0.016
CPT	0.322
CPT+SFT	0.167
SFT Only	0.162

averaged over all L SID layers:

$$\text{RSA} = \frac{1}{L} \sum_{\ell=1}^L \rho(\text{triu}(D^{\text{text},\ell}), \text{triu}(D^{\text{SID},\ell})). \quad (7)$$

A higher RSA indicates that the learned SID space correctly mirrors the semantic topology of the LLM’s preexisting textual knowledge. **Baselines & Training Stages.** To isolate the impact of our vocabulary grounding strategy, we evaluate the Text-Grounding RSA across four distinct configurations:

- **Random Initialization:** The state of the new SID tokens immediately after being appended to the LLM vocabulary. These embeddings are initialized as Gaussian samples with mean equal to the mean of the base text embeddings and small covariance.
- **CPT:** The embeddings after CPT, which trains the frozen LLM on a semantic grounding task using item text descriptions to establish text structure before downstream behavioral tuning.
- **CPT+SFT:** The embeddings after SFT from the CPT checkpoint, where SFT trains only on SID interaction sequences.
- **SFT Only:** The embeddings after SFT from the pretrained Qwen3-0.6B model, without CPT.

Results. As shown in Table 2, new SIDs start with almost no correlation to their text embeddings at initialization (RSA of 0.016). Applying CPT establishes a strong initial semantic alignment, raising the RSA significantly to 0.322. This confirms that the continued pretraining stage successfully anchors the novel SID tokens within the LLM’s pre-existing textual knowledge space. However, this explicit textual grounding is mostly lost after subsequent behavioral tuning. Following SFT on user interaction sequences, the CPT+SFT configuration experiences a stark drop in RSA to 0.167. This pulls it down to a level similar to training with SFT alone (0.162), which itself achieves a non-trivial RSA likely due to natural correlations between item semantics and user co-engagement patterns. This convergence suggests that while CPT effectively injects initial semantic priors, the sequential recommendation objectives of SFT aggressively reshape the embedding space, overriding pure text semantics to prioritize collaborative relationships. Nevertheless, the strong initial grounding provided by CPT may still confer optimization benefits during SFT training. We quantify the downstream retrieval gain due to CPT in §4.3.

4 Retrieval Evaluation

Having evaluated the results of the tokenization and CPT stages, we now evaluate the results of SFT through retrieval performance. We guide our evaluation by four research questions:

- **RQ1 (Offline Quality):** How does SnapLGR perform against the TIGER-style T5 baseline at offline SID retrieval?
- **RQ2 (Improvement Attribution):** Which design choices drive SnapLGR's offline gains?
- **RQ3 (Training and Inference Efficiency):** How much do our training and inference optimizations enhance throughput?
- **RQ4 (Online Impact):** What is the online production impact of the full SnapLGR system?

To our knowledge, we are the first to systematically study the performance gap between LLM-based GR and the TIGER-style T5 baseline at industrial scale. Moreover, our online results demonstrate the practical utility of SnapLGR. We provide additional offline studies, including studying the effects of training data volume and beam width, in Appendix C.

4.1 Offline Training and Evaluation Setup

Data. SFT uses a 1-day window, and evaluation uses the immediately following hour. CPT uses a 1-month window of video data that ends 1 week prior to the SFT training date. Evaluation uses up to 10 future items per request as ground-truth labels.

Metrics. Unless otherwise noted, offline experiments evaluate SID-level generation before materialization. We use Recall@ k (abbreviated R@ k) and Pass@ k (P@ k), and also report Pos-1-Pass@ k in extended results in Appendix C (see also for full metric definitions). We use beam search with width 32, and Recall@32 is our primary offline metric to match the production beam width of 32.

TIGER-style Baseline (T5). Our legacy generative retrieval baseline follows the TIGER [27] paradigm, utilizing a 13M-parameter T5 encoder-decoder architecture initialized from scratch. The deployed production control uses CLIP-based SIDs with coarser codebook widths, (256, 256, 256) rather than our (1024, 512, 256). To cleanly isolate model architecture and capacity, for our offline SID retrieval comparisons we train this baseline using the exact same Qwen3-VL SIDs, training data, and evaluation protocol as our LLM models. We refer to this baseline simply as **T5** hereafter.

4.2 Overall Model Comparison (RQ1)

To address **RQ1**, Table 3 compares the generative retrieval performance of our SnapLGR architecture against the legacy T5 encoder-decoder baseline. By fully harnessing LLM pretraining, continued pretraining (CPT), and supervised fine-tuning (SFT), SnapLGR dramatically outperforms the legacy system across all key top- k retrieval metrics. Most notably, SnapLGR achieves a 2.51 $\times$ increase in Pass@10 (21.78% vs. 8.685%) and a 2.27 $\times$ increase in Pass@32. Similar magnitude improvements are observed in item recall, with Recall@32 reaching 11.11% compared to the baseline's 4.624% (2.40 $\times$ lift). These results underscore the substantial representational and ranking capacity advantages of the pretrained causal LLM architecture over traditional encoder-decoder setups for semantic identifier retrieval.

4.3 Attribution of Improvement Sources (RQ2)

We evaluate three non-mutually-exclusive hypotheses to understand *why* SnapLGR outperforms legacy GR baselines. Because this

Table 3: Offline SID retrieval performance comparison over Qwen3-VL SIDs. SnapLGR yields a greater than 2 $\times$ improvement across all top- k metrics relative to the T5 baseline.

Model	P@10	P@32	R@10	R@32
T5	8.685	15.26	2.672	4.624
SnapLGR	21.78	34.67	6.437	11.11
Ratio (SnapLGR / T5)	2.51 $\times$	2.27 $\times$	2.41 $\times$	2.40 $\times$

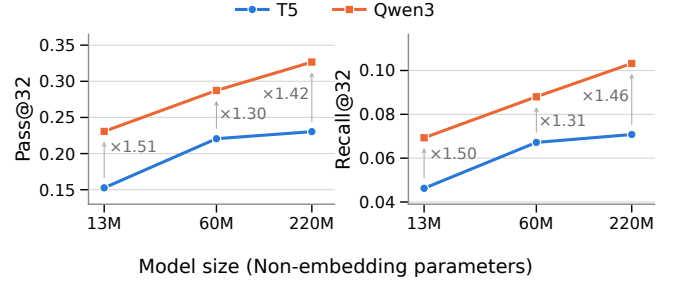


Figure 4: Pass@32 (left) and Recall@32 (right) for Qwen3 and T5 architectures across parameter scales from 13M to 600M. Annotations indicate the relative performance multiplier of Qwen3 compared to T5 at matching scales. All models are trained using SFT only from random initialization.

offline comparison holds the tokenizer fixed, we ablate the tokenizer separately, as a design choice rather than a driver of the gap.

4.3.1 Hypothesis 1: Impact of Model Architecture. Figure 4 compares retrieval performance across randomly initialized decoder-only Qwen3 models and matched T5 encoder-decoder baselines. Note that all experiments use SFT only (no CPT) with random initialization to cleanly isolate structural efficiency. The Qwen3 decoder-only models consistently outperform the T5 baseline on both Pass@32 and Recall@32 at every matched parameter scale. Specifically, Qwen3 delivers a 1.51 $\times$ lift in Pass@32 and a 1.50 $\times$ lift in Recall@32 at the 13M scale, maintaining strong relative advantages ranging from 1.30 $\times$ to 1.46 $\times$ at the 60M and 220M scales. This is the largest effect we isolate, and it does not appear substitutable by capacity alone: T5 at 220M reaches a Pass@32 of 23.03%, close to what Qwen3 attains at 13M (23.07%). This suggests that the decoder-only architecture is inherently better suited for generative retrieval tasks.

4.3.2 Hypothesis 2: Effect of Model Scaling. Using the same experimental setup from Figure 4, we also observe the effect of raw parameter scaling. Both architectures exhibit strong positive scaling behavior, with absolute retrieval performance steadily climbing as the parameter count increases from 13M to 600M. The two families appear to saturate at different points, however: T5 gains +44.6% in Pass@32 from 13M to 60M but only +4.4% from 60M to 220M, whereas Qwen3 still gains +14.3% over that interval. Crucially, the structural advantage of the decoder-only architecture holds robustly across these different model capacities.

Table 4: Benefit of LLM pretraining and grounding. All experiments use Qwen3-0.6B, with either Random or Pre-trained initialization, and Qwen3-VL SIDs.

Model	P@10	P@32	R@10	R@32
(1) Random + SFT	20.88	34.07	6.106	10.77
(2) Pretrained + SFT	21.60	34.46	6.341	10.98
(3) Pretrained + CPT + SFT	21.78	34.67	6.437	11.11
Lift (2) → (3)	+0.83%	+0.62%	+1.51%	+1.19%

Table 5: SID-level and Video-level retrieval recall metrics for Qwen3-0.6 backbones trained with MM, MM+, and Qwen3-VL SIDs (SFT-only).

Tokenizer	SID Recall		Video Recall	
	@10	@32	@10	@32
MM	6.102	10.58	1.393	2.720
MM+	5.979	10.30	1.364	2.616
Qwen3-VL	6.341	10.98	1.416	2.765

4.3.3 H3: Benefit of Pretraining and Grounding. Table 4 isolates the impact of LLM pretraining and continued pretraining (CPT) on downstream performance. Initializing with pretrained weights (*Pretrained + SFT*) provides a clear baseline boost across all metrics compared to training from scratch (*Random + SFT*), improving P@10 from 20.88% to 21.60% and R@10 from 6.106% to 6.341%. Furthermore, incorporating continued pretraining (*Pretrained + CPT + SFT*) yields further gains, achieving the best performance across all metrics (21.78% P@10 and 11.11% R@32). While these gains represent the smallest of the three evaluated effects, they remain consistent and practically relevant in the context of industrial-scale traffic.

4.3.4 SID Tokenizer Ablations. Table 5 isolates the effect of SID tokenization within a single 600M LLM setup. At the SID level, Qwen3-VL SIDs outperform legacy MM and MM+ tokenizers across all metrics (e.g., SID-Recall@32 of 10.98% vs. 10.58% and 10.30%, respectively), and this advantage carries through downstream materialization to video-level retrieval, where Qwen3-VL reaches a Video-Recall@32 of 2.765% against MM (2.720%) and MM+ (2.616%). We note that while Qwen3-VL achieves the best performance in both downstream retrieval and collision resistance (§3.1), the overall correlation between collision metrics and retrieval quality is not perfect. Specifically, the legacy MM tokenizer outperforms MM+ in retrieval despite exhibiting worse collision resistance. This suggests that while strong distributional properties are crucial prerequisites for reliable materialization, they are not absolute predictors of end-to-end performance, and further study is needed to fully disentangle their interplay.

4.4 Training and Inference Efficiency (RQ3)

To address **RQ3**, we evaluate the impact of our system optimizations on training and inference throughput.

Table 6: Compounding training optimizations leading to production deployment. All experiments use a single node with 8 A100 GPUs, besides the last row which uses a single node of 8 H100 GPUs. Speedups are cumulative relative to the un-optimized baseline, measured in terms of samples/s/GPU.

Setting	Speedup
Baseline	1.00×
+ torch.compile	1.29×
+ FA2 Variable length support	1.47×
+ Optimized packed sequences	1.78×
+ H100 GPUs	3.63×

4.4.1 Training optimization. Table 6 details the cumulative impact of our training optimizations on system throughput. Starting from a standard baseline on an 8 × NVIDIA A100-80GB node, applying torch.compile yields an initial 1.29× speedup by reducing framework overhead and fusing GPU operations. We further optimize attention computation by integrating FlashAttention-2 with variable-length support, which raises the relative throughput by 1.47×. We leverage dynamic sequence packing to eliminate wasted computation on padding tokens; by subsequently optimizing the target packed sequence length to maximize hardware utilization, we push the A100 throughput to 1.78×. Finally, migrating this fully optimized software stack to a single node equipped with NVIDIA H100 GPUs provides a substantial hardware-accelerated boost, achieving a peak end-to-end training throughput of 3.63× relative to the baseline. These compounding software and hardware optimizations are critical for enabling SnapLGR’s frequent periodic retraining within strict production budgets.

4.4.2 Inference optimization. Our initial profiling revealed that standard beam search performance was primarily constrained by framework overhead rather than GPU computation. Specifically, Python-based beam scoring, KV-cache forking, coordination, and scheduler overhead dominated execution time. On a multi-GPU setup with beam width 32, actual GPU decoding accounted for only 12–15% of total wall-clock time.

TensorRT-LLM. To address the primary bottleneck, we migrated the Python-based beam search implementation to TensorRT-LLM’s CUDA-backed beam search runtime. This eliminated much of the beam management overhead and resulted in a 9.3× throughput improvement on a single A100 GPU at beam width 32, while maintaining equivalent retrieval accuracy.

Decentralized worker-loop architecture. We redesigned the production inference workflow from a centralized driver-dispatch architecture to a *decentralized worker-loop* processing model to improve distributed inference efficiency. Each worker independently claims, loads, and processes its assigned data shards, effectively removing head-node fan-out bottlenecks. Running this worker-loop architecture on a 64×A100 cluster achieved an additional 3.3× throughput improvement over the centralized architecture.

Table 7: Compounding inference throughput optimizations. Speedups are cumulative relative to the baseline centralized Python-based architecture on A100 GPUs, measured in terms of samples/s/GPU.

Optimization	Speedup
Baseline Centralized Architecture	1.00×
+ TensorRT-LLM CUDA-backed beam search	9.3×
+ Decentralized worker-loop architecture	30.7×
+ Asynchronous I/O and System Overlap	45.7×

Table 8: Online A/B results: SnapLGR vs. legacy T5 baseline.

Metric	Relative Change	p-value
View Time	+0.37% $\pm$ 0.27%	0.007
Time Spent	+0.09% $\pm$ 0.09%	0.048
Deep Sessions	+0.18% $\pm$ 0.16%	0.027
Deep Sessions Unique User	+0.11% $\pm$ 0.09%	0.017

Asynchronous I/O and System Overlap. After optimizing the framework and distribution architecture, generation speed was bottlenecked by data I/O and CPU-side execution. We further optimized the worker-loop by executing background row-group prefetching to hide read latencies, isolating Parquet writes to a dedicated asynchronous pool to hide storage latencies, and disabling the execution of unused metric encoding operations during the generation loop. Together, this combination of I/O and system overlap optimizations provided another significant gain, multiplying throughput by 1.49× relative to the un-optimized worker-loop baseline.

Cumulative System Gains. As detailed in Table 7, integrating these optimizations yielded compounding improvements across the serving stack. End-to-end, migrating from the legacy Python-based centralized architecture to the fully optimized TensorRT-LLM worker-loop delivered an approximate 45.7× increase in cumulative per-GPU throughput on A100 GPUs.

4.5 Online Production Impact (RQ4)

Table 8 reports a 7-day production A/B test of SnapLGR against the legacy T5 baseline. SnapLGR achieved statistically significant positive gains across core feed engagement metrics, highlighted by a +0.37% relative increase in View Time, a +0.09% increase in Time Spent, a +0.18% increase in Deep Sessions, and a +0.11% increase in Deep Sessions Unique User. At our platform’s scale these relative lifts amount to a substantial absolute gain in aggregate engagement hours, and the Deep Session lifts indicate that the improved candidate quality drives sustained, high-intent engagement and return visits rather than merely longer immediate viewing.

We note that this production comparison changes the tokenizer, backbone, and serving stack simultaneously, so the A/B measures the launched system as a whole; §4.3 isolates individual components offline under a fixed tokenizer.

5 Related Work

Generative retrieval and SIDs. DSI [29] and GENRE [1] pioneered mapping queries to document identifiers via autoregressive generation. TIGER [27] brought this paradigm to recommendation, deriving hierarchical SIDs from RQ-VAE [17] codebooks and training a seq2seq Transformer to generate them; Singh et al. [28] further showed these IDs generalize to ranking. Recently, SID construction has been extended to incorporate multimodal signals [37] and collaborative contrastive objectives [30]. Our work extends this thread by applying Personalized PageRank (PPR) signals to supervise the collaborative objective. We also build upon the non-collaborative RQ-VAE design documented in Ju et al. [13], utilizing full-codebook straight-through optimization and layer normalization to mitigate codebook collapse.

LLM-based recommendation at scale. Early frameworks like P5 [6] cast recommendation as text generation, inspiring subsequent efforts to ground pretrained LLMs in collaborative semantics via embedding alignment [20] and instruction tuning [34]. Recently, industrial recommendation systems have rapidly adopted large generative models across sequential transducers, unified retrieve-and-rank architectures, and personalized ranking tasks [4, 5, 7, 11, 21, 22, 24, 31–33, 35, 36]. The closest industrial precursor to our modeling pipeline is PLUM [9], which adapts LLMs for generative retrieval at YouTube using SID tokenization, continued pretraining, and task-specific fine-tuning. While our training methodology closely follows PLUM (§2.2), our work specifically addresses the systems-level challenges of production deployment, introduces PPR-based co-engagement supervision, evaluates SID quality and embedding grounding, and studies the drivers of improvement against conventional TIGER-style GR.

6 Conclusion

We present the end-to-end design, production launch, and empirical evaluation of SnapLGR, an LLM-based GR system for a short video platform at Snapchat. By jointly co-designing multimodal PPR-supervised SIDs, two-stage training including vocabulary grounding, and distributed training and inference, our system achieved significant online lifts in View Time, Time Spent, Deep Sessions, and Deep Sessions Unique User over the incumbent TIGER-style baseline. Our deployment highlights two practical takeaways for practitioners. First, deploying LLM-based GR at industrial scale requires co-optimizing the serving infrastructure; advanced inference techniques, such as CUDA-backed beam search and decentralized worker architectures, are essential to making wide-beam generation practically viable. Second, our offline attribution reveals that transitioning to a decoder-only architecture is the primary driver of the observed retrieval improvements, while enhanced tokenization and vocabulary grounding provide smaller, yet vital, compounding benefits. Together, these findings underscore the need for a holistic approach when scaling modern GR systems. For future work, we plan to explore scaling to larger LLM backbones to leverage their more extensive pretrained world knowledge for GR.

References

- [1] Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. 2021. Autoregressive Entity Retrieval. In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=5k8F6UU39V>

- [2] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [3] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *The Twelfth International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2307.08691>
- [4] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. *arXiv preprint arXiv:2502.18965*. arXiv:2502.18965 [cs.IR] <https://arxiv.org/abs/2502.18965>
- [5] Hamed Firooz, Maziar Sanjabi, Adrian Englund, Aman Gupta, Ben Levine, Dre Olgati, Gungor Polatkan, Iulia Melnychuk, Karthik Ramgopal, Kirill Talanin, Kutta Srinivasan, Luke Simon, Natesh Sivasubramoniapillai, Necip Fazil Ayan, Qingquan Song, Samira Sriram, Souvik Ghosh, Tao Song, Vignesh Kothapalli, Xiaoling Zhai, Ya Xu, Yu Wang, and Yun Dai. 2025. 360Brew: A Decoder-only Foundation Model for Personalized Ranking and Recommendation. *arXiv preprint arXiv:2501.16450*. arXiv:2501.16450 [cs.IR] <https://arxiv.org/abs/2501.16450>
- [6] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5). In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys)*. doi:10.1145/3523227.3546767
- [7] Ruidong Han, Bin Yin, Shangyu Chen, He Jiang, Fei Jiang, Xiang Li, Chi Ma, Mingcong Huang, Xiaoguang Li, Chunzhen Jing, Yueming Han, Menglei Zhou, Lei Yu, Chuan Liu, and Wei Lin. 2025. MTGR: Industrial-Scale Generative Recommendation Framework in Meituan. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*. doi:10.1145/3746252.3761565
- [8] Taher H Haveliwala. 2002. Topic-sensitive pagerank. In *Proceedings of the 11th international conference on World Wide Web*. 517–526.
- [9] Ruining He, Lukasz Heldt, Lichan Hong, Raghunandan Keshavan, Shifan Mao, Nikhil Mehta, Zhengyang Su, Alicia Tsai, Yueqi Wang, Shao-Chuan Wang, Xinyang Yi, Lexi Baugher, Baykal Kakici, Ed Chi, Cristos Goodrow, Ningren Han, He Ma, Romer Rosales, Abby Van Soest, Devansh Tandon, Su-Lin Wu, Weilong Yang, and Yilin Zheng. 2026. PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*. doi:10.1145/3774904.3792802
- [10] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training Compute-Optimal Large Language Models. In *Advances in Neural Information Processing Systems*.
- [11] Yanhua Huang, Yuqi Chen, Xiong Cao, Rui Yang, Mingliang Qi, Yinghao Zhu, Qingchang Han, Yaowei Liu, Zhaoyu Liu, Xuefeng Yao, Yuting Jia, Leilei Ma, Yinqi Zhang, Taoyu Zhu, Liujie Zhang, Lei Chen, Weihang Chen, Min Zhu, Ruiwen Xu, and Lei Zhang. 2025. Towards Large-scale Generative Ranking. *arXiv preprint arXiv:2505.04180*; presented at GenAIRecP @ KDD 2025. arXiv:2505.04180 [cs.IR] <https://arxiv.org/abs/2505.04180>
- [12] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvish Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6420–6425.
- [13] Clark Mingxuan Ju, Tong Zhao, Leonardo Neves, Liam Collins, Bhuvish Kumar, Jiwen Ren, Lili Zhang, Wenfeng Zhuo, Vincent Zhang, Xiao Bai, Jinchao Li, Karthik Iyer, Zihao Fan, Yilun Xu, Yiwen Chen, Peicheng Yu, Manish Malik, and Neil Shah. 2026. Semantic IDs for Recommender Systems at Snapchat: Use Cases, Technical Challenges, and Design Choices. *arXiv preprint arXiv:2604.03949* (2026). arXiv:2604.03949 [cs.IR]
- [14] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*. arXiv:2001.08361 [cs.LG]
- [15] Nikolaus Kriegeskorte, Marieke Mur, and Peter A Bandettini. 2008. Representational similarity analysis-connecting the branches of systems neuroscience. *Frontiers in systems neuroscience* 2 (2008), 4.
- [16] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, et al. 2022. Matryoshka representation learning. *Advances in Neural Information Processing Systems* 35 (2022), 30233–30249.
- [17] Doyup Lee, Chihyeon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive Image Generation using Residual Quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 11523–11532.
- [18] Houyi Li, Wenzheng Zheng, Jingcheng Hu, Qiufeng Wang, Hanshan Zhang, Zili Wang, Yangshijie Xu, Shuigeng Zhou, Xiangyu Zhang, and Daxin Jiang. 2025. Predictable Scale: Part I – Optimal Hyperparameter Scaling Law in Large Language Model Pretraining. arXiv:2503.04715 [cs.LG] <https://arxiv.org/abs/2503.04715>
- [19] Mingxin Li, Yanzhao Zhang, Dingkun Long, Keqin Chen, Sibao Song, Shuai Bai, Zhibo Yang, Pengjun Xie, An Yang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2026. Qwen3-VL-Embedding and Qwen3-VL-Reranker: A Unified Framework for State-of-the-Art Multimodal Retrieval and Ranking. *arXiv preprint arXiv:2601.04720* (2026). arXiv:2601.04720 [cs.CV] <https://arxiv.org/abs/2601.04720>
- [20] Jiayi Liao, Sihang Li, Zhengyi Yang, Jiancan Wu, Yancheng Yuan, Xiang Wang, and Xiangnan He. 2024. LLaRA: Large Language-Recommendation Assistant. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. 1785–1795.
- [21] Qingyun Liu, Bo Yan, Yang Liu, Yuji Roh, Ekansh Sharma, Likang Yin, Emma Olowo, Min hsuan Tsai, Yuxuan Li, Diego Uribe, Saksham Aggarwal, Siqi Wu, Yuan Hao, Vikas Kedigehalli, Lukasz Heldt, Lichan Hong, Li Wei, and Xinyang Yi. 2026. TokenMinds: Pretrained User Tokens and Embeddings for User Understanding in Large Recommender Systems. *arXiv preprint arXiv:2606.25147*. arXiv:2606.25147 [cs.IR]
- [22] Zhanyu Liu, Shiyao Wang, Xingmei Wang, Rongzhou Zhang, Jiaxin Deng, Honghui Bao, Jinghao Zhang, Wuchao Li, Pengfei Zheng, Xiangyu Wu, Yifei Hu, Qigen Hu, Xinchun Luo, Lejian Ren, Zixing Zhang, Qianqian Wang, Kuo Cai, Yunfan Wu, Hongtao Cheng, Zexuan Cheng, Lu Ren, Huanjie Wang, Yi Su, Ruiming Tang, Kun Gai, and Guorui Zhou. 2025. OneRec-Think: In-Text Reasoning for Generative Recommendation. *arXiv preprint arXiv:2510.11639*. arXiv:2510.11639 [cs.IR]
- [23] NVIDIA Corporation. 2023. TensorRT-LLM: A TensorRT Toolbox for Optimized Large Language Model Inference. GitHub repository, <https://github.com/NVIDIA/TensorRT-LLM>.
- [24] OneRec Team, Biao Yang, Boyang Ding, Chenglong Chu, Dunju Zang, Fei Pan, Han Li, Hao Jiang, Honghui Bao, Huanjie Wang, Jian Liang, Jiangxia Cao, Jiao Ou, Jiaxin Deng, Jinghao Zhang, Kun Gai, Lu Ren, Peiru Du, Pengfei Zheng, Rongzhou Zhang, Ruiming Tang, Shiyao Wang, Siyang Mao, Siyuan Lou, Teng Shi, Wei Yuan, Wenlong Xu, Xingchen Liu, Xingmei Wang, Xinqi Jin, Yan Sun, Yan Wang, Yifei Hu, Yingzhi He, Yufei Ye, Yuhao Wang, Yunhao Zhou, Yuqin Dai, Zhao Liu, Zhipeng Wei, Zhixin Ling, Ziming Li, Zixing Zhang, Ziyuan Liu, et al. 2026. OneReason Technical Report. *arXiv preprint arXiv:2606.06260*. arXiv:2606.06260 [cs.IR] <https://arxiv.org/abs/2606.06260>
- [25] Qwen Team. 2025. Qwen3-0.6B. <https://huggingface.co/Qwen/Qwen3-0.6B>. Hugging Face model card.
- [26] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67. <https://www.jmlr.org/papers/v21/20-074.html>
- [27] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, Maciej Kula, Ed Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*.
- [28] Anima Singh, Trung Vu, Nikhil Mehta, Raghunandan Keshavan, Maheswaran Sathiamoorthy, Yilin Zheng, Lichan Hong, Lukasz Heldt, Li Wei, Devansh Tandon, Ed H. Chi, and Xinyang Yi. 2024. Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems (RecSys)*. ACM, Bari, Italy. doi:10.1145/3640457.3688190
- [29] Yi Tay, Vinh Q. Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, Tal Schuster, William W. Cohen, and Donald Metzler. 2022. Transformer Memory as a Differentiable Search Index. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.
- [30] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable Item Tokenization for Generative Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, Boise, ID, USA, 2400–2409. doi:10.1145/3627673.3679569
- [31] Yuhao Yang, Zhi Ji, Zhaopeng Li, Yi Li, Zhonglin Mo, Yue Ding, Kai Chen, Zijian Zhang, Jie Li, Shuanglong Li, and Lin Liu. 2025. Sparse Meets Dense: Unified Generative Recommendations with Cascaded Sparse-Dense Representations. In *Advances in Neural Information Processing Systems*, Vol. 38. https://papers.nips.cc/paper_files/paper/2025/hash/86ba836d4c5dd859d795a172911745e2-Abstract-Conference.html
- [32] Chao Yi, Dian Chen, Gaoyang Guo, Jiakai Tang, Jian Wu, Jing Yu, Mao Zhang, Sunhao Dai, Wen Chen, Wenjun Jiang, Yuning Jiang, Zhuji Gao, Bo Zheng, et al. 2025. RecGPT Technical Report. *arXiv preprint arXiv:2507.22879*. arXiv:2507.22879 [cs.IR] <https://arxiv.org/abs/2507.22879>

- [33] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 58484–58509.
- [34] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting Large Language Models by Integrating Collaborative Semantics for Recommendation. In *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*.
- [35] Guorui Zhou, Honghui Bao, Jiaming Huang, Jiaxin Deng, Jinghao Zhang, Junda She, Kuo Cai, Lejian Ren, Lu Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rongzhou Zhang, Ruiming Tang, Shiyao Wang, Wuchao Li, Xiangyu Wu, Xinchun Luo, Xingmei Wang, Yifei Hu, Yunfan Wu, Zhanyu Liu, Zhiyang Zhang, Zixing Zhang, et al. 2025. OpenOneRec Technical Report. arXiv preprint arXiv:2512.24762. arXiv:2512.24762 [cs.IR]
- [36] Guorui Zhou, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Shiyao Wang, Weifeng Ding, Wuchao Li, Xinchun Luo, Xingmei Wang, Zexuan Cheng, Zixing Zhang, et al. 2025. OneRec Technical Report. arXiv preprint arXiv:2506.13695. arXiv:2506.13695 [cs.IR] <https://arxiv.org/abs/2506.13695>
- [37] Jing Zhu, Mingxuan Ju, Yozen Liu, Danai Koutra, Neil Shah, and Tong Zhao. 2025. Beyond Unimodal Boundaries: Generative Recommendation with Multimodal Semantics. arXiv preprint arXiv:2503.23333. arXiv:2503.23333 [cs.IR] <https://arxiv.org/abs/2503.23333>

A Contributions

Author roles are as follows:

- **Core Contributors:** Liam Collins, Jiwen Ren
- **Contributors:** Donald Loveland, Bhuvish Kumar, Clark Mingxuan Ju, Xuan Guo, Mo Li, Alvin Hou, Yi Cui, Peng Yang, Jian Wang, Saud Afzal Shafi, Nga Than, Ruiming Lu
- **Project Lead:** Wenfeng Zhuo, Dongheng Li
- **Leadership:** Lili Zhang, Mingtao Zhang, Jinchao Ye, Vincent Xue, Chunhui Zhu, Neil Shah

B Additional Implementation Details

B.1 SID Pipeline

We use the architecture and straight-through estimator from Ju et al. [13]; Section 2.1 gives the mathematical formulation. Codebooks are orthogonally initialized and L2-normalized. The implementation applies a softmax to the cosine scores to form the backward-pass surrogate in Equation (2).

We use the first 1024 dimensions of the ambient 4096-dimensional Qwen3-VL-Embedding-8B embedding. This is possible because the model is trained with Matryoshka representation learning [16], meaning embedding prefixes are also valid embeddings.

B.1.1 Personalized PageRank (PPR) Formulation. To compute the co-engagement signal for SID construction (Section 2.1), we rely on Personalized PageRank (PPR) [8]. For each anchor video i , we extract its two-hop neighborhood from the bipartite user–video engagement graph and form a row-normalized transition matrix $\mathbf{P}$, where P_{ab} is the probability of moving uniformly from node a to one of its neighbors. The PPR score vector π_i is the stationary solution to:

$$\pi_i = (1 - \alpha)\mathbf{e}_i + \alpha\mathbf{P}^\top \pi_i, \quad (8)$$

where $\mathbf{e}_i$ places all mass on anchor i and α is the transition probability. Equivalently, a walk starts at i and repeatedly either returns to i with probability $1 - \alpha$ or moves to a random neighboring

CPT Prompt:

Video <|sid_begin|><s_a_857><s_b_412><s_c_10><|sid_end|>
is about a person dressed as a clown sitting on a ledge.
A person walks up to the clown and they hug. The video
is in black and white.

Figure 5: Illustrative CPT training prompt. The video corpus grounds SID tokens in textual semantics.

node with probability α . This mechanism ensures the resulting similarity scores remain centered on the anchor’s local neighborhood rather than drifting toward globally popular regions of the graph.

B.2 LLM Training Pipeline

B.2.1 CPT Prompt. Figure 5 gives an illustrative CPT prompt.

B.3 Inference Pipeline

B.3.1 Value-weighted materialization. We maintain an SID-to-Video ID lookup table to enable fast materialization of videos given generated SIDs. Our lookup table retains up to a fixed number of candidate stories per SID. Within each SID bucket, stories are ranked using a multi-objective linear value function that weighs downstream engagement signals:

$$V(i) = \sum_{m \in \mathcal{M}} w_m \cdot S_m(i), \quad (9)$$

where $S_m(i)$ represents the estimated rate for metric m and w_m is the corresponding weight that favors stronger engagement signals.

C Additional Experimental Details and Results

This appendix reports the complete SID-level metric grids for the offline analyses in Section 4. All values are percentages. Unless noted otherwise, the backbone retrieval model is the pretrained-initialized Qwen3-0.6B SFT-only 600k-step checkpoint and inference uses beam width 32. We also provide additional details of the metrics and report the results of new experiments in the following subsections.

C.1 SID Quality Metrics

We provide further details on the metrics we use for our collision resistance study in Section 3.1. The following metrics are defined over a matched item corpus $\mathcal{I}$:

- **Utilization:** The proportion of the theoretical SID code space actively occupied by videos, defined as $|\mathcal{Z}_{\text{obs}}|/\min(|\mathcal{I}|, \prod_{\ell=1}^L K_\ell)$, where $\mathcal{Z}_{\text{obs}}$ is the set of realized SIDs and K_ℓ is the codebook size at level ℓ .
- **Uniqueness:** The percentage of videos in the corpus whose complete assigned SID contains exactly one video.
- **Top-1% Coverage:** An interpretable snapshot of extreme head concentration, measured by sorting realized SIDs by bucket size (descending) and calculating the fraction of total corpus videos contained within the largest 1% of SID buckets.
- **Gini Coefficient:** A holistic summary of the collision distribution shape. It is calculated as the area between the cumulative

Table 9: Full performance comparison between the legacy T5 encoder-decoder baseline and the reference SnapLGR.

Metric	T5	SnapLGR	Ratio
Pass@1	1.711	4.857	2.84×
Pass@5	5.608	14.78	2.63×
Pass@10	8.685	21.78	2.51×
Pass@32	15.26	34.67	2.27×
Pos-1-Pass@1	0.762	2.051	2.70×
Pos-1-Pass@5	2.317	6.057	2.61×
Pos-1-Pass@10	3.469	8.853	2.55×
Pos-1-Pass@32	5.778	14.41	2.49×
Recall@1	0.556	1.318	2.36×
Recall@5	1.749	4.195	2.40×
Recall@10	2.672	6.437	2.41×
Recall@32	4.624	11.11	2.40×

Table 10: Complete SID-level metric grid for the SID-type comparison of Table 5 (models trained with MM, MM+, and Qwen3-VL SIDs).

Metric	MM	MM+	Qwen3-VL
Pass@1	4.719	4.616	4.896
Pass@5	14.29	13.94	14.74
Pass@10	21.02	20.49	21.60
Pass@32	33.60	32.76	34.46
Pos-1-Pass@1	1.987	1.951	2.082
Pos-1-Pass@5	5.800	5.672	6.053
Pos-1-Pass@10	8.504	8.293	9.022
Pos-1-Pass@32	13.88	13.45	14.38
Recall@1	1.279	1.263	1.331
Recall@5	4.005	3.938	4.170
Recall@10	6.102	5.979	6.341
Recall@32	10.58	10.30	10.98

video coverage curve (Figure 3) and a perfectly uniform diagonal. A lower Gini score indicates that videos are evenly balanced across assigned SIDs, whereas a higher score (approaching 1.0) reveals severe concentration into a few massive “mega-collision” codes.

C.2 Retrieval Metrics

Let $\mathcal{T}_r = \text{set}(\hat{\mathcal{T}}_r)$ be the ground-truth target SIDs for request r , and let $\hat{\mathcal{T}}_{r,k}$ consist of the model’s top- k generated SIDs. Recall@ k is defined as $|\mathcal{T}_r \cap \hat{\mathcal{T}}_{r,k}|/|\mathcal{T}_r|$. Pass@ k is 1 if $\mathcal{T}_r \cap \hat{\mathcal{T}}_{r,k}$ is not empty and 0 otherwise, and Pos-1-Pass@ k (short for Position-1-Pass@ k) indicates whether $\hat{\mathcal{T}}_{r,k}$ contains the immediate next ground-truth SID $\mathbf{z}_{r,b_r+1}$. Metrics are averaged uniformly across samples and given as percentages.

Table 11: SFT (SID-only, from random initialization) retrieval eval across downscaled Qwen3 sizes. Sizes are backbone (non-embedding) params.

Metric	13M	60M	220M
Pass@1	2.60	3.56	4.28
Pass@5	8.49	11.01	13.19
Pass@10	13.05	16.77	19.62
Pass@32	23.07	28.73	32.66
Pos-1-Pass@1	1.10	1.53	1.83
Pos-1-Pass@5	3.28	4.39	5.37
Pos-1-Pass@10	4.94	6.54	7.99
Pos-1-Pass@32	8.48	11.13	13.38
Recall@1	0.80	1.04	1.19
Recall@5	2.51	3.14	3.76
Recall@10	3.83	4.89	5.80
Recall@32	6.93	8.80	10.32

C.3 Hyperparameter Tuning

Model scaling learning-rate selection. In the model scaling experiment, learning rates for the T5 retrievers were initially estimated with Step’s Law [18] anchored from the previously-tuned learning rate of $1e-3$ at 13M parameters, and then tuned to as small as 5×10^{-5} , decreasing with model width; the Qwen3 runs reuse the resulting per-size values. The model size, tuned learning rate pairs are: (13M, 1×10^{-3}), (60M, 3.4×10^{-4}), (220M, 2.6×10^{-4}).

Table 12: Effect of training data scale. Below gives metrics across checkpoints of an SFT-only (no CPT) training run. Columns give training progress as a fraction of one epoch.

Metric	0.2	0.4	0.6	0.8	1.0
Tokens (B)	69.1	138.2	207.4	276.5	345.6
Pass@1	3.89	4.37	4.60	4.81	4.90
Pass@5	12.00	13.27	13.87	14.54	14.74
Pass@10	17.82	19.54	20.38	21.34	21.60
Pass@32	28.93	31.28	32.60	34.12	34.46
Pos-1-Pass@1	1.60	1.83	1.94	2.03	2.08
Pos-1-Pass@5	4.70	5.33	5.65	5.93	6.05
Pos-1-Pass@10	6.90	7.80	8.25	8.67	8.85
Pos-1-Pass@32	11.27	12.63	13.38	14.13	14.38
Recall@1	1.08	1.20	1.26	1.31	1.33
Recall@5	3.39	3.75	3.93	4.10	4.17
Recall@10	5.14	5.69	5.97	6.24	6.34
Recall@32	8.85	9.77	10.27	10.83	10.98

C.4 Training Data Scaling

Table 12 evaluates five checkpoints from one pretrained-initialized Qwen3-0.6B SFT-only run; no CPT is used. Recall improves monotonically at every cutoff: from 0.2 to 1 epoch, Recall@32 rises from 8.85% to 10.98% (+2.126 percentage points, or +24.0% relative). The

final 0.2-epoch interval adds only 0.149 points, suggesting that gains taper at the end of the observed range. The Pass@32 and Pos-1-Pass@32 metrics also rise, from 28.93% to 34.46% and from 11.27% to 14.38%, respectively.

Table 13: Complete metrics for the initialization comparison of Table 4 with both runs SFT-only, no CPT. Deltas are pre-trained relative to random-init.

Metric	Random	Pretrained	Δ (%)
Pass@1	4.578	4.896	+6.9%
Pass@5	14.08	14.74	+4.7%
Pass@10	20.88	21.60	+3.4%
Pass@32	34.07	34.46	+1.1%
Pos-1-Pass@1	1.936	2.082	+7.5%
Pos-1-Pass@5	5.724	6.053	+5.7%
Pos-1-Pass@10	8.441	8.847	+4.8%
Pos-1-Pass@32	14.00	14.38	+2.7%
Recall@1	1.259	1.331	+5.7%
Recall@5	3.986	4.170	+4.6%
Recall@10	6.106	6.341	+3.8%
Recall@32	10.77	10.98	+1.9%

Table 14: Effect of beam width on SID-level metrics for baseline SFT-only checkpoint.

Metric	Beam 32	Beam 96	Δ (pp)	Δ (%)
Pass@1	4.90	4.90	+0.008	+0.16
Pass@5	14.74	14.85	+0.113	+0.77
Pass@10	21.60	22.02	+0.416	+1.93
Pass@32	34.46	37.90	+3.448	+10.01
Pos-1-Pass@1	2.08	2.09	+0.003	+0.13
Pos-1-Pass@5	6.05	6.10	+0.045	+0.74
Pos-1-Pass@10	8.85	9.02	+0.173	+1.96
Pos-1-Pass@32	14.38	16.03	+1.654	+11.50
Recall@1	1.33	1.33	-0.000	+0.07
Recall@5	4.17	4.19	+0.022	+0.53
Recall@10	6.34	6.44	+0.101	+1.59
Recall@32	10.98	12.25	+1.269	+11.56

C.5 Beam Width Latency-Recall Trade-off.

Table 14 shows that increasing the beam width from 32 to 96 leads to substantial gains for metrics with larger k , for example Recall@32 increased by 11.56%. However, moving from width 32 to 96 reduced throughput by 45.0% on A100 GPUs.

C.6 SID Quality Case Study

To qualitatively assess the semantic fidelity of our Qwen3-VL-based multimodal and co-engagement-supervised tokenization pipeline,

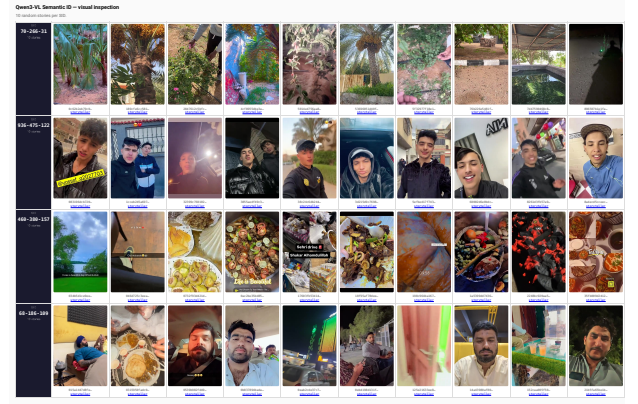


Figure 6: Example SID-to-video mappings. Each row contains images from 10 randomly-selected videos with the same SID. Videos with the same SID are highly semantically-similar.

we visually inspect videos that are mapped to the same Semantic ID (SID). Figure 6 illustrates several example SID-to-video mappings, where each row displays keyframes from 10 randomly selected videos sharing an identical complete SID. As demonstrated in the figure, videos assigned to the same SID exhibit strong visual and topical coherence. This visual consistency confirms that the multimodal, behavior-infused RQ-VAE successfully preserves content semantics while compressing dense item embeddings into discrete identifier buckets.